

Locust makes it easy to run load tests distributed over multiple machines. It is event-based (using [gevent](#)), which makes it possible for a single process to handle many thousands of concurrent users. While there may be other tools that are capable of doing more requests per second on a given hardware, the low overhead of each Locust user makes it very suitable for testing highly concurrent workloads.

Even though Locust primarily works with websites/services, it can be used to test almost any system or protocol. Just [write a client](#) for what you want to test, or [explore some created by the community](#).

To run the test file: **locust -f simple_test.py**

To access the dashboard: <http://localhost:8089/>

Simple test file with 2-tasks:

```
from locust import HttpUser, task

class HelloWorldUser(HttpUser):
    @task
    def hello_world(self):
        self.client.get('/hello')

    @task
    def hello_world_02(self):
        self.client.get('/world')
```

To run Locust in headless mode:

locust --headless --users 10 --spawn-rate 1 -H <http://your-server.com>

```

import time
from locust import HttpUser, task, between

class QuickStartUser(HttpUser):
    wait_time = between(1, 10)

    @task
    def hello_world(self):
        self.client.get("/hello")
        self.client.get("/world")

    # maximize the chance to choose this task by 3-times
    @task(3)
    def view_items(self):
        for item_id in range(10):
            self.client.get(f"/item?id={item_id}", name="/item")
            time.sleep(1)

    def on_start(self):
        self.client.post("/login", json={"username": "foo", "password": "bar"})

```

HttpUser is not a real browser, and thus will not parse an HTML response to load resources or render the page. It will keep track of cookies, though.

A User's [wait_time](#) method makes it easy to introduce delays after each task execution. If no *wait_time* is specified, the next task will be executed as soon as one finishes.

- `constant` for a fixed amount of time
- `between` for a random time between a min and max value

For example, to make each user wait between 0.5 and 10 seconds between every task execution:

```
from locust import User, task, between

class MyUser(User):
    @task
    def my_task(self):
        print("executing my_task")

        wait_time = between(0.5, 10)
```

- `constant_throughput` for an adaptive time that ensures the task runs (at most) X times per second.
- `constant_pacing` for an adaptive time that ensures the task runs (at most) once every X seconds (it is the mathematical inverse of *constant_throughput*).

Note

For example, if you want Locust to run 500 task iterations per second at peak load, you could use `wait_time = constant_throughput(0.1)` and a user count of 5000.

Wait time can only constrain the throughput, not launch new Users to reach the target. So, in our example, the throughput will be less than 500 if the time for the task iteration exceeds 10 seconds.

Wait time is applied *after* task execution, so if you have a high spawn rate/ramp up you may end up exceeding your target during ramp-up.

Wait times apply to *tasks*, not requests. For example, if you specify `wait_time = constant_throughput(2)` and do two requests in your tasks, your request rate/RPS will be 4 per User.

It's also possible to declare your own `wait_time` method directly on your class. For example, the following `User` class would sleep for one second, then two, then three, etc.

```
class MyUser(User):
    last_wait_time = 0

    def wait_time(self):
        self.last_wait_time += 1
        return self.last_wait_time

    ...
```

weight and fixed_count attributes

If more than one user class exists in the file, and no user classes are specified on the command line, Locust will spawn an equal number of each of the user classes. You can also specify which of the user classes to use from the same locustfile by passing them as command line arguments:

```
$ locust -f locust_file.py WebUser MobileUser
```

If you wish to simulate more users of a certain type than another you can set a `weight` attribute on those classes. The code below will make Locust spawn 3 times as many `WebUsers` as `MobileUsers`:

```
class WebUser(User):
    weight = 3
    ...

class MobileUser(User):
    weight = 1
    ...
```

Also, you can set the `fixed_count` attribute. In this case, the weight attribute will be ignored and only that exact number users will be spawned. These users are spawned before any regular, weighted ones. In the example below, only one instance of `AdminUser` will be spawned, to make some specific work with more accurate control of request count independently of total user count.

```
class AdminUser(User):
    wait_time = constant(600)
    fixed_count = 1

    @task
    def restart_app(self):
        ...

class WebUser(User):
    ...
```

environment attribute

A reference to the `environment` in which the user is running. Use this to interact with the environment, or the `runner` which it contains. E.g. to stop the runner from a task method:

```
self.environment.runner.quit()
```

If run on a standalone locust instance, this will stop the entire run. If run on worker node, it will stop that particular node.

on_start and on_stop methods

Users (and [TaskSets](#)) can declare an [on_start](#) method and/or [on_stop](#) method. A User will call its [on_start](#) method when it starts running, and its [on_stop](#) method when it stops running. For a TaskSet, the [on_start](#) method is called when a simulated user starts executing that TaskSet, and [on_stop](#) is called when the simulated user stops executing that TaskSet (when [interrupt\(\)](#) is called, or the user is killed).

When a load test is started, **an instance of the User class will be created for each simulated user**, and they will start running within their own greenlet. When these users run, they pick tasks that they execute, sleep for a while, and then pick a new task, and so on.

tasks attribute

Another way to define the tasks of a User is by setting the `tasks` attribute.

The `tasks` attribute is either a list of Tasks, or a `<Task : int>` dict, where Task is either a python callable or a `TaskSet` class. If the task is a normal python function they receive a single argument which is the User instance that is executing the task.

Here is an example of a User task declared as a normal python function:

```
from locust import User, constant

def my_task(user):
    pass

class MyUser(User):
    tasks = [my_task]
    wait_time = constant(1)
```

If the tasks attribute is specified as a list, each time a task is to be performed, it will be randomly chosen from the `tasks` attribute. If however, `tasks` is a dict - with callables as keys and ints as values - the task that is to be executed will be chosen at random but with the int as ratio. So with a task that looks like this:

```
{my_task: 3, another_task: 1}
```

`my_task` would be 3 times as likely to be executed as `another_task`.

@tag decorator

By tagging tasks using the `@tag` decorator, you can be picky about what tasks are executed during the test using the `--tags` and `--exclude-tags` arguments. Consider the following example:

```
from locust import User, constant, task, tag

class MyUser(User):
    wait_time = constant(1)

    @tag('tag1')
    @task
    def task1(self):
        pass

    @tag('tag1', 'tag2')
    @task
    def task2(self):
        pass

    @tag('tag3')
    @task
    def task3(self):
        pass

    @task
    def task4(self):
        pass
```

If you started this test with `--tags tag1`, only *task1* and *task2* would be executed during the test. If you started it with `--tags tag2 tag3`, only *task2* and *task3* would be executed.

`--exclude-tags` will behave in the exact opposite way. So, if you start the test with `--exclude-tags tag3`, only *task1*, *task2*, and *task4* will be executed. Exclusion always wins over inclusion, so if a task has a tag you've included and a tag you've excluded, it will not be executed.

init

The `init` event is triggered at the beginning of each Locust process. This is especially useful in distributed mode where each worker process (not each user) needs a chance to do some initialization. For example, let's say you have some global state that all users spawned from this process will need:

```
from locust import events
from locust.runners import MasterRunner

@events.init.add_listener
def on_locust_init(environment, **kwargs):
    if isinstance(environment.runner, MasterRunner):
        print("I'm on master node")
    else:
        print("I'm on a worker or standalone node")
```

Other events

See [extending locust using event hooks](#) for other events and more examples of how to use them.

client attribute / HttpSession

`client` is an instance of `HttpSession`. `HttpSession` is a subclass/wrapper for `requests.Session`, so its features are well documented and should be familiar to many. What `HttpSession` adds is mainly reporting of the request results into Locust (success/fail, response time, response length, name).

It contains methods for all HTTP methods: `get`, `post`, `put`, ...

Just like `requests.Session`, it preserves cookies between requests so it can easily be used to log in to websites.

Make a POST request, look at the response and implicitly reuse any session cookie we got for a second request

```
response = self.client.post("/login", {"username":"testuser", "password":"secret"})
print("Response status code:", response.status_code)
print("Response text:", response.text)
response = self.client.get("/my-profile")
```

`HttpSession` catches any `requests.RequestException` thrown by `Session` (caused by connection errors, timeouts or similar), instead returning a dummy `Response` object with `status_code` set to 0 and `content` set to `None`.

Validating responses

Requests are considered successful if the HTTP response code is OK (<400), but it is often useful to do some additional validation of the response.

You can mark a request as failed by using the `catch_response` argument, a *with*-statement and a call to `response.failure()`

```
with self.client.get("/", catch_response=True) as response:
    if response.text != "Success":
        response.failure("Got wrong response")
    elif response.elapsed.total_seconds() > 0.5:
        response.failure("Request took too long")
```

You can also mark a request as successful, even if the response code was bad:

```
with self.client.get("/does_not_exist/", catch_response=True) as response:
    if response.status_code == 404:
        response.success()
```

You can even avoid logging a request at all by throwing an exception and then catching it outside the with-block. Or you can throw a [locust exception](#), like in the example below, and let Locust catch it.

```
from locust.exception import RescheduleTask
...
with self.client.get("/does_not_exist/", catch_response=True) as response:
    if response.status_code == 404:
        raise RescheduleTask()
```